МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №2
по дисциплине «Объектно-ориентированное программирование»
Тема: Создание консольной игры

Студент гр. 5381 _____ Лопатин Д. И.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург
2026

Цель работы.

Расширить архитектуру консольной игры с добавлением системы способностей роботов, системы модификаторов, накладываемых на роботов и клетки поля, инвентаря предметов игрока, а также модификацией визуализатора.

Задание.

Основные требования (6 баллов):

- Создать класс-интерфейс способности робота, которые он может применять. Класс-интерфейса не должен ничего хранить, содержать только публичные чисто виртуальные методы.

  Сделать реализацию интерфейса, создав классы следующих способностей:

  - Удар вокруг - нанести урон всем роботам в радиусе 1-й клетки.
  - Дальний удар - нанести урон одному выбранному роботу в радиусе 3-х клеток.
  - Лечение - восстановить фиксированное количество здоровья роботу, использующему способность.
  - Телепортация - переместиться на выбранную клетку в радиусе 5-и клеток, если она является проходимой и не занята другим роботом.
  - Каждая способность требует энергию для использования. В начале игры, игрок выбирает начальную способность, которая будет у него доступна сразу. Вражеские роботы получают случайно способность Лечение или Телепортация. За один ход, робот может применить способность сколько угодно раз, пока есть энергия для их использования.
- Модифицировать повышение ранга. У робота игрока при повышении ранга дополнительно должна быть возможность, выбрать еще одну способность (если не все получены), либо улучшить, которая

имеется. Улучшения: Удар вокруг - увеличение радиуса, Дальний удар - увеличение урона, Лечение - увеличение лечения, Телепортация - снижение количества необходимой энергии.

- Создать класс-интерфейс статуса. Статусы применяются к роботам, поэтому нужно добавить методы для работы и обработки статусов. Реализовать следующие статусы:
    - Замедление - запрещает передвижение на один ход, статус пропадает в конце хода. При получении нескольких замедлений, скорость уменьшается на несколько ходов.
    - Перегрузка - запрещает использование способности, статус пропадает при попытке использовать способность (способность не активируется). При получении нескольких перегрузок, должно срабатывать как одна перегрузка.
    - Горение - наносит урон в течение нескольких ходов (количество ходов задается через аргумент конструктора статуса), статус пропадает спустя указанное количество ходов. При получении нескольких горений, продлевается длительность статуса.
    - Щит - блокирует фиксированное количество единиц урона, статус спадает при получении урона больше значения щита (разница в уроне в таком случае наносится по здоровью). При получении нескольких щитов, увеличивается количество блокируемого урона.
- Добавить к клеткам игрового поля возможность накладывать статусы. При генерации игрового поля, клетка может получить от 0 до 2 статусов, которые накладываются на робота, когда он наступает на клетку. Чем больше статусов на клетке, там меньше вероятность появления таких клеток. После того, как робот наступил на клетку, убирается один случайный статус, который она может наложить.

Дополнительные требования (2 балла за каждое требование):

- Модифицировать способности следующим образом:
    - Удар вокруг - дополнительно накладывает статус Перегрузка.
    - Дальний удар - дополнительно накладывает статус Горение.
    - Лечение - дополнительно накладывает статус Щит при попытке вылечить здоровье больше максимального. Размер щита равен значению, на которое лечение превышает максимальное.
- Модифицировать систему визуализации, добавив режим переключения отображения. Стандартный, подсветка врагов, подсветка непроходимых клеток, подсветка клеток, накладывающих статусы (для CLI подсветка - выделение определенным цветом).

Выполнение работы.

UML-диаграмма программы представлена на рис. 1.

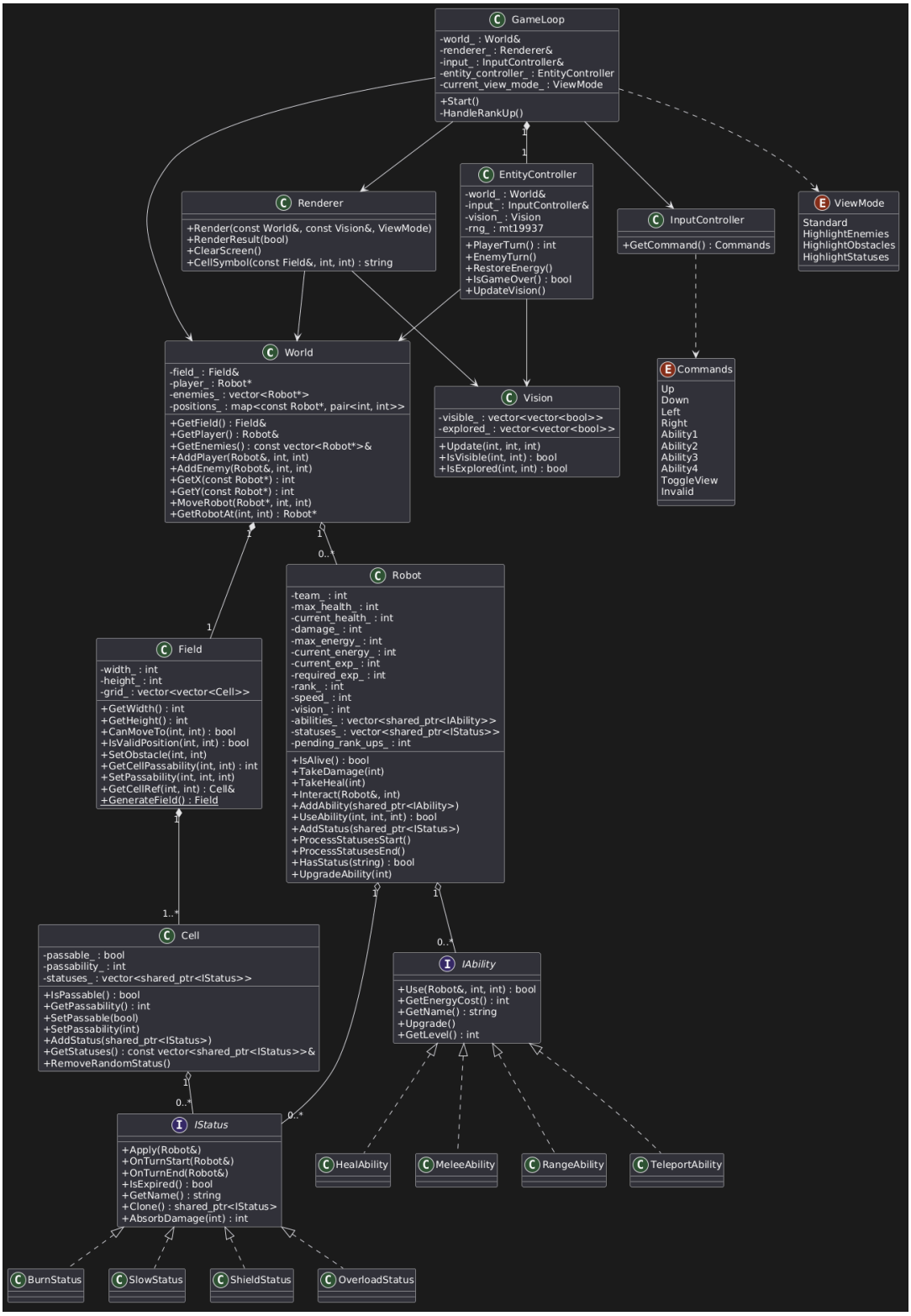


Рисунок 1 – UML-диаграмма.

1. Описание архитектуры.

1.1. Общая концепция.

В рамках лабораторной работы архитектура системы была расширена подсистемами способностей, временных модификаторов и реализацией :

1. **Подсистема способностей** построена на интерфейсе IAbility. Он не хранит никаких данных, а только задает общие правила: сколько энергии стоит способность, как её применить и как улучшить. Все 4 способности (Лечение, Удар вокруг, Дальний удар, Телепортация) хранятся в роботе в виде умных указателей. Робот не знает, какая именно способность у него в руках, он просто вызывает общий метод Use. Это позволяет легко добавлять новые способности в будущем, не меняя код самого робота.

2. **Подсистема статусов (эффектов)** работает через интерфейс IStatus. Статусы могут висеть как на роботах, так и лежать на клетках поля в виде ловушек. Они влияют на игру по-разному: Замедление запрещает ходить, Перегрузка блокирует использование способностей, Горение наносит урон каждый ход, а Щит поглощает входящий урон. Для ловушек на карте используется метод Clone(), который создает копию статуса. Это нужно, чтобы при наступании на клетку робот получал свой личный статус, а оригинальная ловушка оставалась лежать на карте.

3. **Система улучшения способностей** реализована через хранение списка способностей внутри робота. За убийство врагов дается опыт. Когда опыта набирается достаточно, робот повышает ранг. В игровом цикле (GameLoop) при этом вызывается меню, где игрок может либо выучить новую способность, либо улучшить уже имеющуюся (увеличить урон, радиус, количество лечения или снизить затраты энергии).

4. **Система визуализации** была расширена для удобства игрока. Класс Renderer теперь поддерживает переключение режимов отображения (ViewMode). С помощью кнопки V можно включить подсветку определенных объектов (врагов, стен или ловушек). Для этого используются ANSI-цвета, которые окрашивают нужные символы на карте. Кроме того, справа от карты появилась информационная панель, которая в реальном времени показывает здоровье, энергию и текущие активные статусы игрока (включая оставшуюся прочность щита).

1.2. Изменения в архитектуре и их обоснование.

Перед добавлением нового функционала (способностей и статусов) в архитектуру первой лабораторной работы были внесены изменения. Изначально классы были слишком сильно связаны друг с другом, что мешало масштабированию. Были проведены следующие улучшения:

1. **Введение класса** World**.** Изначально GameLoop и EntityController хранили у себя одинаковые ссылки на поле, игрока и список врагов. Это было дублирование данных. Мы создали класс World, который стал единым центром хранения состояния игры. Теперь контроллеры обращаются к миру, а не к отдельным сущностям.

2. **Вынос координат из класса** Robot **в** World**.** Раньше робот сам хранил свои координаты x и y. Логически это неверно: сущность (робот) не должна знать, где она находится на карте, это задача игрового пространства. Хранение координат было вынесено из Robot в класс World. Для этого был добавлен контейнер positions_, который сопоставляет каждого робота с его текущей позицией на карте. Теперь вся информация о местоположении объектов сосредоточена в World.

3. **Развязка** Vision **и** Field**.** Класс тумана войны (Vision) раньше принимал в метод обновления ссылку на поле, чтобы проверить границы. Мы убрали эту зависимость. Теперь Vision знает свои размеры и проверяет границы самостоятельно. Если в будущем изменится класс Field, нам не придется менять код Vision.

4. **Разделение перегруженного метода** TryMove**.** Метод перемещения был слишком большим (около 45 строк) и делал всё сразу: считал направление, проверял стены, обрабатывал столкновения и двигал робота. Мы разбили его на три маленьких вспомогательных метода: GetDelta (получение сдвига), CanStep (проверка проходимости) и ApplyStep (применение шага). Это сделало код метода TryMove коротким и читаемым.

5. **Добавление валидации.** В классы были добавлены проверки. Например, Robot теперь не может быть создан с отрицательным здоровьем или уроном, а World не позволяет создать робота за пределами карты или на стене. Это защищает игру от случайных поломок.

Эти изменения снизили связность компонентов и повысили их автономность. Проведенный рефакторинг позволил легко интегрировать полиморфные подсистемы способностей (IAbility) и статусов (IStatus) во второй лабораторной работе, сохранив целостность основной архитектуры.

2. Описание классов.

2.1. Подсистема способностей.

Для реализации способностей, которые используют роботы, были разработаны следующие классы:

Класс *IAbility:* интерфейс способности

Назначение класса: абстрактный класс, который задает общий контракт для всех способностей. Не содержит полей и хранит только чисто виртуальные методы.

Публичные методы:

- *virtual bool Use(Robot& caster, int target_x = -1, int target_y = -1) = 0;* применение способности. Принимает ссылку на того, кто использует способность, а также координаты цели (если они требуются).
- *virtual int GetEnergyCost() const = 0;* получение стоимости использования способности в энергии.
- *virtual std::string GetName() const = 0;* получение названия способности.
- *virtual void Upgrade() = 0;* улучшение способности при повышении ранга робота.
- *virtual int GetLevel() const = 0;* получение текущего уровня прокачки способности.

Класс *HealAbility*: способность «Лечение».

Назначение: восстанавливает здоровье робота, использующего способность. При улучшении увеличивается количество восстанавливаемых HP.

Класс *MeleeAbility*: способность «Удар вокруг».

Назначение: наносит урон всем врагам в радиусе действия. Хранит ссылку на World для поиска врагов вокруг. При улучшении увеличивается радиус атаки и накладывается статус «Перегрузка».

Класс *RangeAbility*: способность «Дальний удар».

Назначение: наносит урон одной выбранной цели на расстоянии. Хранит ссылку на World для проверки наличия цели на клетке. При улучшении увеличивается урон и накладывается статус «Горение».

Класс *TeleportAbility*: способность «Телепортация».

Назначение: перемещает робота на выбранную свободную клетку в радиусе действия. Хранит ссылку на World для проверки проходимости клетки и отсутствия на ней других роботов. При улучшении снижается стоимость использования в энергии.

2.2. Подсистема статусов.

Данный модуль реализует временные эффекты (статусы), которые могут накладываться на роботов, а также храниться на клетках поля в виде ловушек. Для реализации этой логики были разработаны следующие классы:

Класс *IStatus:* интерфейс статуса.

Назначение класса: абстрактный класс, задающий общий контракт для всех временных эффектов. Не содержит полей, хранит только чисто виртуальные методы.

Публичные методы:

- *virtual void Apply(Robot& target) = 0;* вызывается один раз в момент наложения статуса на робота.
- *virtual void OnTurnStart(Robot& target) = 0;* логика, срабатывающая в начале хода робота (например, нанесение периодического урона).
- *virtual void OnTurnEnd(Robot& target) = 0;* логика конца хода (уменьшение длительности действия эффекта).
- *virtual bool IsExpired() const = 0;* проверяет, закончилось ли действие статуса (нужно для его удаления).
- *virtual std::string GetName() const = 0;* получение названия статуса.

- *virtual std::shared_ptr<IStatus> Clone() const = 0;* создание точной копии статуса. Необходимо для того, чтобы при наступании на клетку-ловушку робот получал личную копию эффекта, а оригинал оставался на карте.
- *virtual int AbsorbDamage(int damage);* метод для перехвата и поглощения входящего урона (по умолчанию просто возвращает урон без изменений).

• **Класс** *BurnStatus:* статус «Горение».

Назначение: наносит фиксированный урон здоровью робота в начале каждого хода. Имеет длительность, по истечении которой исчезает.

• **Класс** *SlowStatus:* статус «Замедление».

Назначение: запрещает роботу перемещаться по полю. При проверке движения в контроллере робот с этим статусом пропускает ход. Имеет длительность.

• **Класс** *ShieldStatus:* статус «Щит».

Назначение: блокирует входящий урон. Переопределяет метод AbsorbDamage, поглощая часть урона за счет своей прочности. Если урон превышает прочность щита, остаток проходит в здоровье.

• **Класс** *OverloadStatus:* статус «Перегрузка».

Назначение: блокирует использование любых способностей. Если робот пытается применить способность, она не срабатывает, а статус сразу же снимается.

• **Интеграция с игровым полем** (класс Cell)**.**

Класс Cell содержит вектор статусов. При генерации карты некоторые клетки получают от 0 до 2 случайных статусов. Когда робот наступает на такую клетку, вызывается метод Clone() для каждого статуса, и его копии накладываются на робота. После этого с клетки удаляется один случайный статус (метод RemoveRandomStatus), поэтому ловушка срабатывает ограниченное число раз.

2.3. Подсистема визуализации.

Данный модуль отвечает за отрисовку игрового процесса в консоли. Все классы игровой логики не знают о том, как они выглядят, весь вывод сосредоточен в отдельном классе Renderer. Для реализации этого модуля были разработаны следующие компоненты:

• **Перечисление** *ViewMode*: режимы отображения карты.

Назначение: позволяет переключать визуальные режимы для удобства игрока. Значения: Standard (обычный вид), HighlightEnemies (подсветка врагов), HighlightObstacles (подсветка стен) и HighlightStatuses (подсветка клеток со статусами).

• **Класс** *Renderer*: средство визуализации.

Назначение Класса: формирование и вывод кадра игры на экран. Класс не хранит состояние игры, а при каждом вызове получает данные из World и Vision и отрисовывает их.

Публичные методы:

- *void Render(const World& world, const Vision& vision, ViewMode mode);* основной метод отрисовки. Рисует рамку карты, символы клеток, роботов, а также правую информационную панель.
- *void RenderResult(bool player_won);* выводит сообщение о победе или поражении по завершении игры.
- *void ClearScreen()*; очищает консоль перед отрисовкой нового кадра с помощью ANSI-последовательностей.
- *std::string CellSymbol(const Field& field, int x, int y);* определяет символ для конкретной клетки (стена, рельеф, ловушка) и возвращает его вместе с пробелом для идеального выравнивания сетки.

Особенности реализации визуализации:

1. **Отображение тумана войны.** В цикле отрисовки Renderer обращается к объекту Vision. Если клетка не исследована, выводится пустое простран-

ство. Если исследована, но не видна сейчас, выводится только рельеф (без врагов и игрока).

2. **Цветовая подсветка (ANSI).** Для выделения объектов используются ANSI-escape последовательности. При выборе определенного режима (ViewMode) меняется цвет символов: враги подсвечиваются ярко-красным, стены синим, а клетки с ловушками фиолетовым. В стандартном режиме игрок отображается зеленым, а ловушки желтым.
3. **Боковая панель статистики.** Справа от карты в реальном времени выводится информация об игроке: текущее и максимальное здоровье, урон, ранг, прогресс опыта, энергия, а также список активных статусов (например, Горение или Щит - 10HP). Там же отображается текущий режим отображения и подсказки по управлению.
4. **Символы ловушек.** Клетки, на которых лежат статусы, имеют уникальные обозначения: S (Щит), ! (Горение), % (Замедление) и ? (Перегрузка). Это позволяет игроку заранее видеть опасные или полезные зоны на исследованной территории.

2.4. Организация проекта и контроль версий.

Для удобства разработки, поддержки и масштабирования кода была выстроена четкая структура папок, а также настроены инструменты сборки и контроля версий.

1. **Структура каталогов.**

Чтобы избежать скопления десятков файлов в одной папке, исходный код был разделен:

- В папке include хранятся все заголовочные файлы (.hpp), в папке src файлы реализации (.cpp). Это классический подход, который разделяет интерфейсы классов и их реализацию.
- Крупные подсистемы были вынесены в отдельные подпапки. В частности, были созданы папки abilities и statuses (внутри include и src), где лежат классы способностей и статусов соответственно. Это делает структу-

ру проекта чистой: сразу видно, где находится логика игры, а где дополнительные модули.

2. **Контроль версий (Git и GitHub).**

   Разработка проекта велась с использованием системы контроля версий Git, а сам репозиторий размещен на GitHub. Это позволило:

- Отслеживать историю изменений. Каждый крупный этап (например, добавление полиморфных способностей, внедрение статусов, рефакторинг архитектуры) фиксировался отдельным коммитом с понятным описанием.
- Безопасно вносить изменения. Если в процессе рефакторинга или добавления новой функциональности что-то ломалось, всегда можно было откатиться к предыдущей стабильной версии кода.
- Возможность удаленного доступа к проекту. Во время отсутствия доступа к стационарному компьютеру выполнение лабораторной работы происходило на других устройствах, что положительно сказалось на скорости реализации работы.

3. Тестирование.

Приложение запускается и работает корректно, игра ожидает ввода игрока, изначально выбрана стандартная подсветка (см. рис. 2).

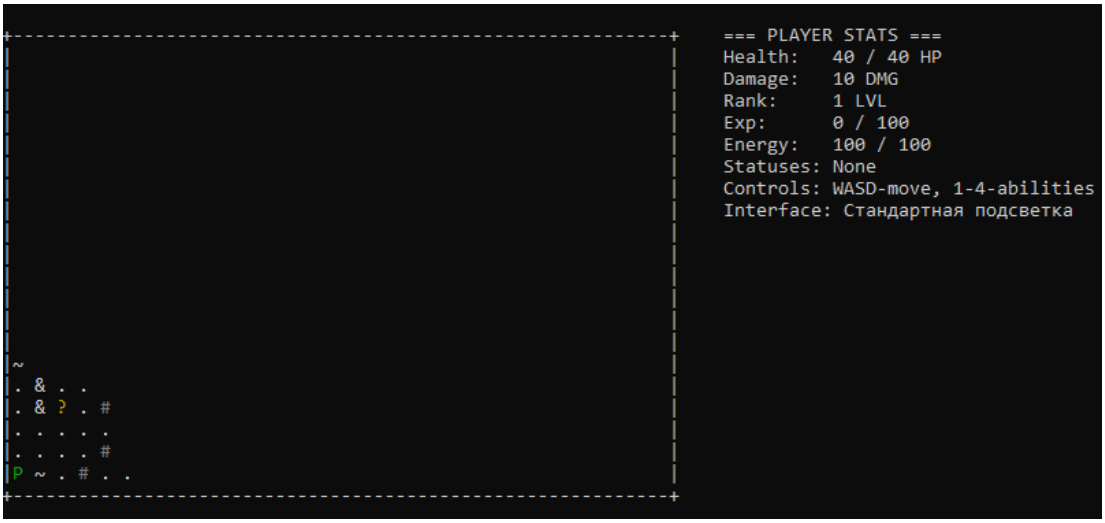


Рисунок 2 – Запуск игры.

Выбор/улучшение способности работает корректно (см. рис. 3, 4).

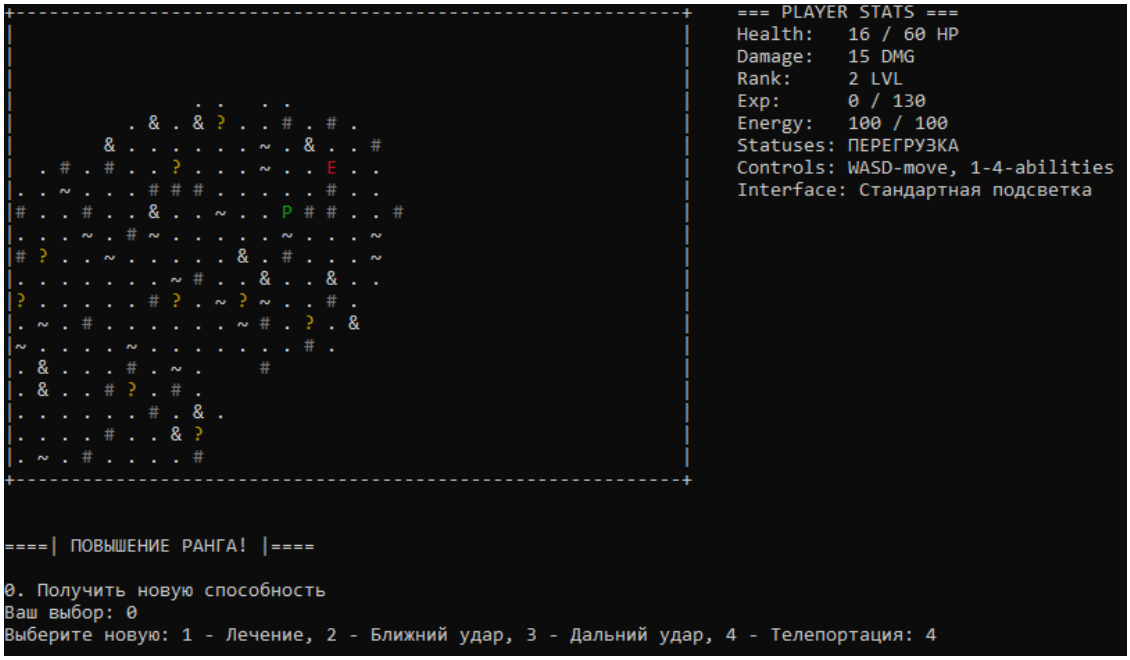


Рисунок 3 – Выбор первой способности.

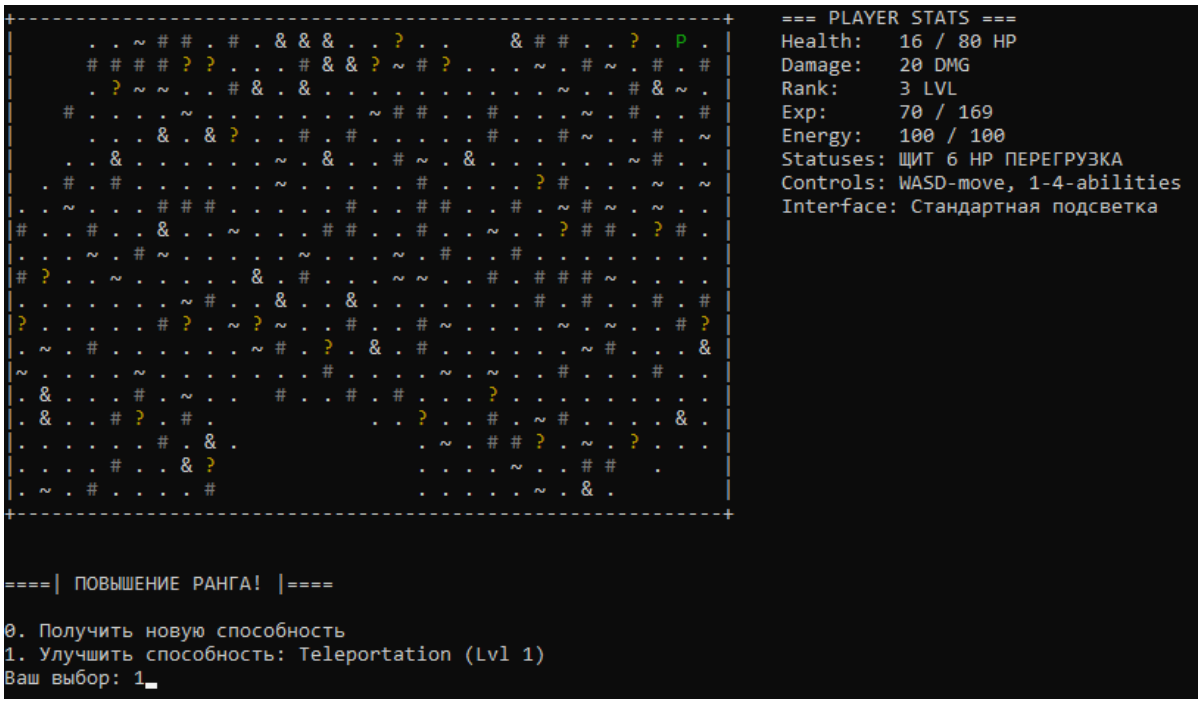


Рисунок 4 – Повышение ранга и выбор улучшения.

Выбор клетки для использования способности работает корректно (см. рис 5, 6)

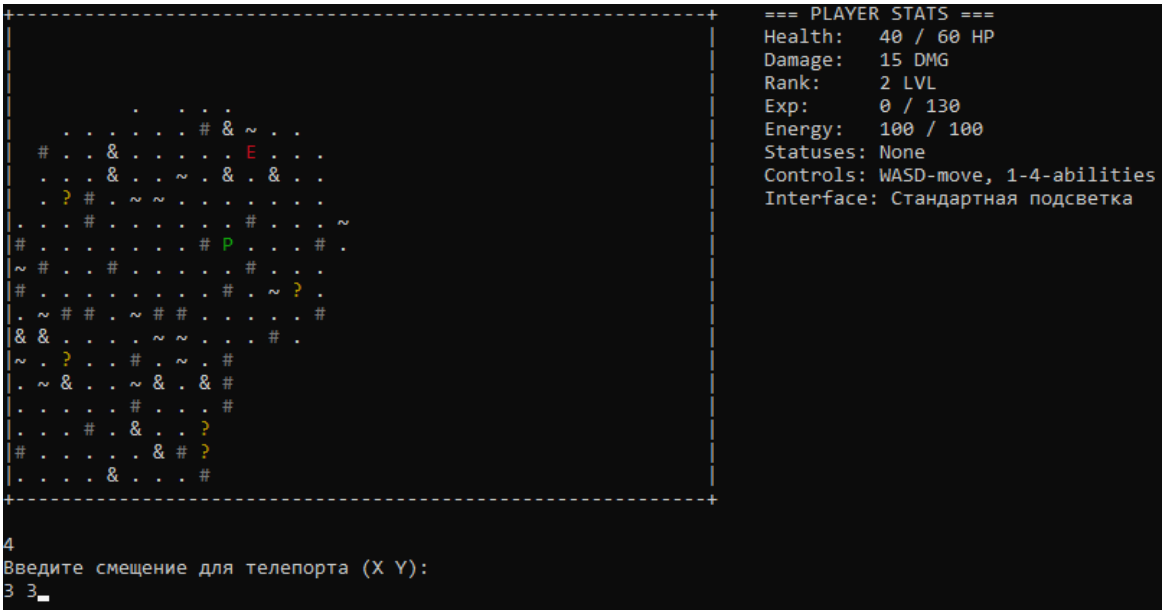


Рисунок 5 – Выбор места использования способности.

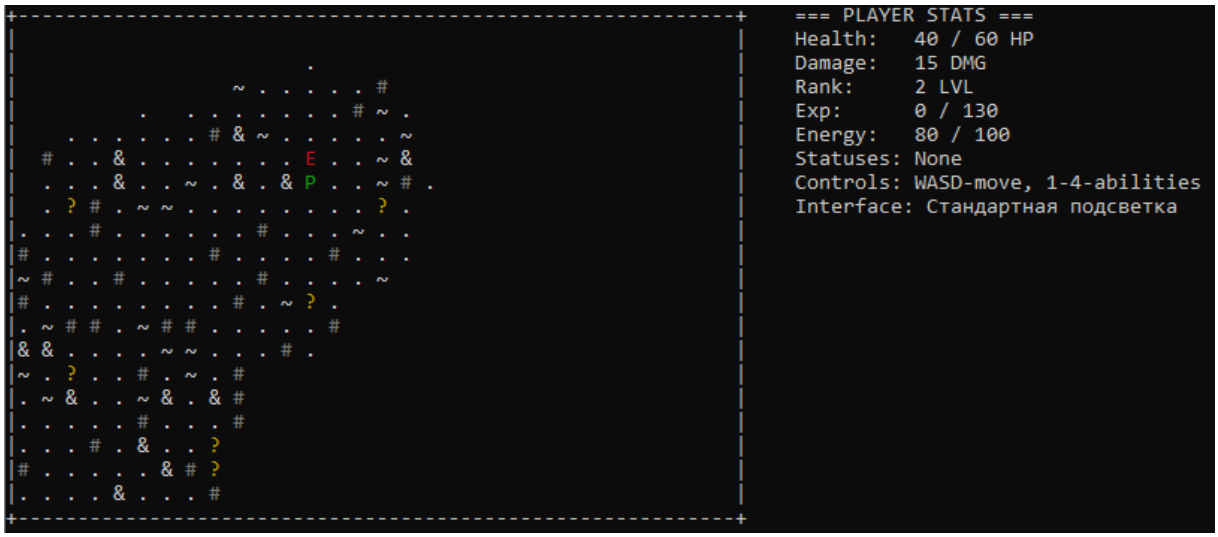
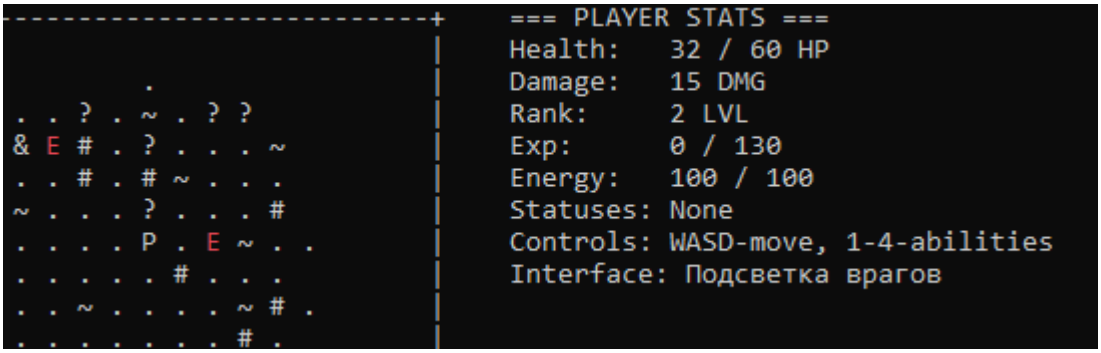
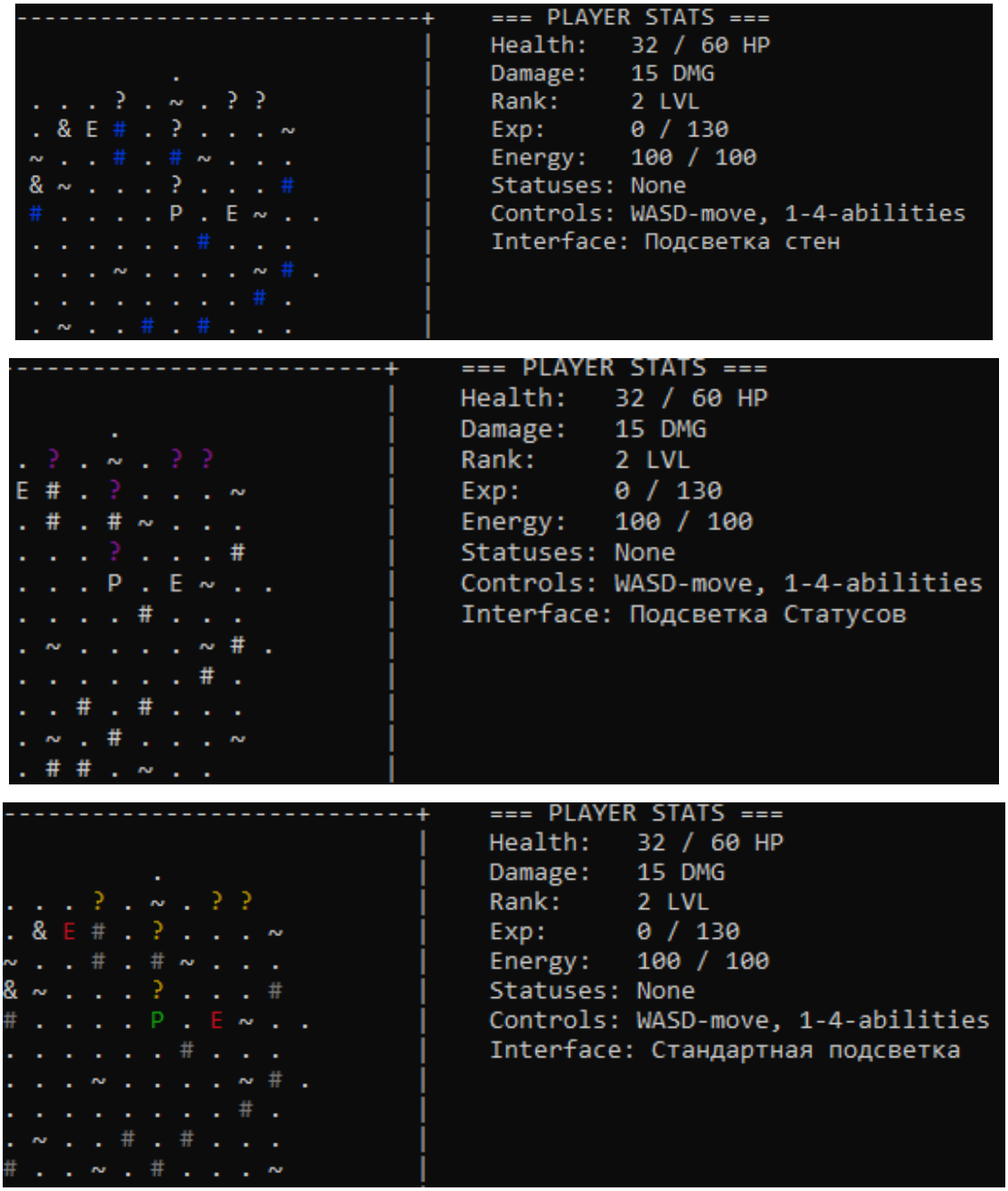


Рисунок 6 – Корректная работа выбора точки телепортации.

Подсветка карты работает корректно (см. рис. 7, 8, 9).





Рисунки 7, 8, 9 – Корректная работа режимов подсветки карты.

Выводы.

В ходе выполнения лабораторной работы №2 архитектура тактической игры на C++ была расширена и оптимизирована. Был внедрен класс World для централизации данных и снижения связности, а также проведен рефакторинг игровой логики.

Реализованы полиморфные подсистемы способностей (IAbility) и статусов (IStatus), которые влияют на роботов и хранятся на клетках в виде ловушек. Игровой процесс дополнен системой энергии, расширенной картой, меню про-

качки рангов и улучшенным интерфейсом с панелью статистики и цветовой подсветкой.

Все основные и дополнительные задачи выполнены в полном объеме. Приложение отвечает стандартам ООП и готово к дальнейшему масштабированию.